WHAT IT IS

ToneTunner is an Android voice input app that runs local Whisper ONNX inference for speech transcription and optional translation to English. It can run as a standalone app, an Input Method Editor (IME), and an Android RecognitionService endpoint.

WHO IT IS FOR

Primary persona: Android users who want offline multilingual speech-to-text and voice input integration in keyboard and system voice-input flows.

WHAT IT DOES

- Records microphone audio at 16 kHz mono PCM with optional WebRTC VAD auto-stop.
- Runs on-device Whisper inference through ONNX Runtime sessions.
- Provides standalone transcription UI with language selector and copy action.
- Supports optional translation mode and optional Text-to-Speech playback.
- Provides IME voice typing and commits recognized text to active input fields.
- Supports speech intent entry point via RECOGNIZE_SPEECH activity.
- Supports system-wide voice input via Android RecognitionService.

KEY ENTRY POINTS

- MainActivity: standalone transcription and translation UI.
- WhisperInputMethodService: IME keyboard voice input.
- WhisperRecognitionService: system speech service integration.
- WhisperRecognizeActivity: popup recognizer for speech intents.
- SetupActivity: model download link and ZIP import workflow.

NOTES

- Inference appears local/on-device in this repo.
- App requests RECORD_AUDIO and related foreground service permissions.

HOW IT WORKS (COMPACT ARCHITECTURE)

Components
- Capture: Recorder + RecordBuffer
- Orchestration: Whisper (threaded task loop, chunk handling)
- Inference: Recognizer + ONNX Runtime + TensorUtils
- Long audio: AudioChunker + ChunkMerger
- Universal model support classes present: ModelManager, ModelConfig, AudioFeatureExtractor, BPETokenizer

Data flow
1. UI/service starts Recorder.
2. Recorder captures PCM audio (optional VAD) and stores bytes in RecordBuffer.
3. Whisper reads normalized float samples from RecordBuffer.
4. Long audio is split into 30-second chunks and merged after per-chunk inference.
5. Whisper invokes Recognizer with language/action.
6. Recognizer executes initializer -> encoder -> cache init -> decoder loop -> detokenizer and returns text/language via callbacks.
7. Caller displays text, commits text in IME, or returns recognition results.

Current engine status
- Dynamic/universal classes are present in repo.
- Active load path in Whisper currently uses legacy Recognizer constructor and helper ONNX files (initializer/cache initializer/detokenizer).
- ModelManager contains TODO for model preference selection wiring.

HOW TO RUN (MINIMAL)
1. Build/install Android app module :app.
2. Launch app; setup opens if model files are missing.
3. Download model ZIP and install via SetupActivity.
4. Use main record button, or enable ToneTunner IME / voice input service.

NOT FOUND IN REPO
- Formal benchmark results (latency/WER/device matrix).
- Explicit automated ASR test suite.
- End-to-end production runbook beyond in-app setup flow.